

Compte rendu TP : Le Responsive Design

Nom : Matisse Kra

Classe : BTS SIO 1ère année

Sommaire

1. Analyse du problème	2
2. Conception de la solution	2
Exercice 1 – Les tailles en pourcentage	2
Exercice 2 – Min-width et Max-width	2
Exercice 3 – Mélange contraintes et tailles relatives	3
Exercice 4 – Flexbox, flex-shrink et flex-grow	3
Exercice 5 – Test d’affichage sur mobile	3
Exercice 6 – La balise meta viewport	3
Exercice 7 – Les media queries	4
Exercice 8 – Application des points de rupture	4
Exercice 9 – Création du menu burger	5
3. Conclusion	6
4. Résultat	6
5. Annexe code CSS final	8

1. Analyse du problème

L'objectif de ce TP est d'apprendre à rendre un site web adaptatif pour qu'il s'affiche correctement sur tous les types d'appareils : ordinateurs, tablettes, smartphones et même smartwatch.

Ce travail permet de découvrir les principes du responsive design, notamment :

- l'utilisation des tailles relatives (%),
- les contraintes min-width et max-width,
- les flexbox pour adapter la mise en page,
- et surtout les media queries pour changer l'apparence selon la taille de l'écran.

Le but final est de rendre le site de Chuck Norris lisible, fluide et ergonomique sur tous les supports.

2. Conception de la solution

Exercice 1 – Les tailles en pourcentage

Dans un premier temps, j'ai donné au <body> une largeur de 100% et une marge automatique pour centrer le contenu :

body {

```
background-color: #838892;
width: 100%; /* largeur demandée */
margin: auto; /* centre le body dans la fenêtre */
}
```

Puis, j'ai mis les largeurs de <article> et <aside> en **pourcentage (67% / 33%)** afin qu'elles s'adaptent à la taille de la fenêtre. Cela permet à la page de se redimensionner automatiquement selon la largeur du navigateur.

Exercice 2 – Min-width et Max-width

Pour éviter que le texte devienne trop étroit ou trop large, j'ai ajouté des limites. Ainsi, le contenu reste lisible même sur les très petits ou très grands écrans.

```
article {
  width: 300px;
  max-width: 500px;
  min-width: 200px;
  flex-shrink: 2; /* rétrécit 2x plus vite qu'un aside */
  flex-grow: 3; /* gagne 3 parts d'espace */
  background-color: #ccc;
  padding: 10px;
}

aside {
  width: 200px;
  max-width: 250px;
  min-width: 150px;
  flex-shrink: 1; /* rétrécit moins vite */
  flex-grow: 1; /* gagne 1 part d'espace */
  background-color: #ccc;
  padding: 10px;
}
```

Exercice 3 – Mélange contraintes et tailles relatives

J'ai testé le comportement des éléments avec les pourcentages et les limites de largeur en redimensionnant la fenêtre.

J'ai observé que lorsque la page devient trop large, certaines divisions cessent de s'étirer car elles sont limitées par leur max-width.

Cela m'a permis de comprendre le concept d'overconstraint (contrainte excessive).

Exercice 4 – Flexbox, flex-shrink et flex-grow

Pour améliorer la mise en page, j'ai utilisé Flexbox dans la balise <main>

```
main {
  display: flex;          /* transforme main en conteneur flex */
  flex-direction: row;    /* met les enfants côte à côte */
  justify-content: space-between; /* laisse de l'espace si nécessaire */
}
```

J'ai ensuite appliqué les propriétés flex-grow et flex-shrink. Grâce à cela, lorsque la fenêtre s'élargit, <article> gagne trois fois plus d'espace que <aside>. Et lorsqu'elle se réduit, <article> diminue plus vite pour garder une mise en page équilibrée. Cela m'a permis de comprendre comment répartir dynamiquement l'espace entre les éléments d'un conteneur.

```
article {
  width: 300px;
  max-width: 500px;
  min-width: 200px;
  flex-shrink: 2; /* rétrécit 2x plus vite qu'un aside */
  flex-grow: 3;   /* gagne 3 parts d'espace */
  background-color: #ccc;
  padding: 10px;
}

aside {
  width: 200px;
  max-width: 250px;
  min-width: 150px;
  flex-shrink: 1; /* rétrécit moins vite */
  flex-grow: 1;   /* gagne 1 part d'espace */
  background-color: #ccc;
  padding: 10px;
}
```

Exercice 5 – Test d'affichage sur mobile

Avec l'outil **Device Mode** de Chrome (F12), j'ai simulé un **Samsung Galaxy S5**. J'ai constaté que le site apparaissait très petit, car le navigateur utilisait une **largeur virtuelle de 980px**. C'est le comportement par défaut des navigateurs mobiles : ils zooment en arrière pour faire rentrer toute la page.

Exercice 6 – La balise meta viewport

Pour corriger cela, j'ai ajouté la ligne suivante dans le <head> :

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Grâce à cette balise, le navigateur affiche la page à la taille réelle de l'écran. Le rendu devient immédiatement plus lisible et plus fidèle à la version responsive.

Exercice 7 – Les media queries

Les **media queries** permettent d'appliquer des styles différents selon la taille de l'écran. J'ai étudié leur syntaxe :

```
@media (max-width: 768px)
```

Elles sont très utiles pour cacher des éléments, changer l'ordre d'affichage ou adapter la mise en page automatiquement.

Exercice 8 – Application des points de rupture

J'ai appliqué trois niveaux de **points de rupture** :

- **en dessous de 768px** : cacher les tableaux,
- **en dessous de 480px** : mettre <article> et <aside> en colonne,
- **en dessous de 168px** : n'afficher que les citations (cas smartwatch).

```
@media (max-width: 768px) {  
  table {  
    display: none; /* masque les tableaux */  
  }  
}  
  
@media (max-width: 480px) {  
  main {  
    flex-direction: column; /* empile les sections */  
  }  
  
  nav {  
    display: none;  
  }  
  
  .burger {  
    visibility: visible;  
    position: absolute;  
    top: 10px;  
    right: 20px;  
  }  
}  
  
@media (max-width: 168px) {  
  aside>h2.impair, ul{  
    display: none;  
  }  
}
```

Grâce à ces conditions, le site devient **totalelement adaptatif**

Exercice 9 – Création du menu burger

Enfin, j'ai ajouté un **menu burger** qui apparaît uniquement sur mobile.

J'ai créé un div `.burger` contenant une image de burger et un menu caché `#menu2`.

```
<div class="burger">
  
  <div id="menu2">
    <div><a href="./index.html">Accueil</a></div>
    <div><a href="./facts.html">Facts</a></div>
    <div><a href="./news.html">Actualités</a></div>
    <div><a href="./contact.html">Contact</a></div>
  </div>
</div>
```

```

  .burger {
    position: relative;          /* base pour le menu interne */
    display: flex;
    justify-content: flex-end;  /* place le bouton à droite */
    align-items: center;
    cursor: pointer;
    visibility: hidden;        /* caché par défaut */
  }
  #menu2 {
    position: fixed;            /* se place par rapport à la fenêtre */
    top: 0;
    right: 0;
    width: 80vw;                /* 80% de la largeur de la fenêtre */
    height: 100vh;
    display: none;              /* masqué par défaut */
    flex-direction: column;
    justify-content: center;
    align-items: center;
    z-index: 1000;
  }
  /* Les liens dans le menu burger */
  #menu2 div {
    margin: 15px 0;
  }

  #menu2 a {
    color: white;
    font-size: 1.5em;
    text-decoration: none;
  }
  /* Quand on survole le burger, le menu s'affiche */
  .burger:hover #menu2 {
    display: flex;
  }
```

Sur les petits écrans, le menu classique `<nav>` disparaît et laisse place à ce **menu latéral plein écran** (80% de la largeur et 100% de la hauteur).

Le menu est positionné en haut à droite avec `position: fixed` et `z-index: 1000` pour qu'il soit au-dessus du reste du contenu.

Cela permet d'avoir une **navigation claire et compacte sur smartphone**.

```
@media (max-width: 480px) {
  main {
    flex-direction: column; /* empile les sections */
  }

  nav {
    display: none;
  }
}
```

```
.burger {  
  visibility: visible;  
  position: absolute;  
  top: 10px;  
  right: 20px;  
}
```

3. Conclusion

Ce TP m'a permis de comprendre les **bases du responsive design** et de **maîtriser les media queries**.

J'ai appris à :

- utiliser les **unités relatives (%)**,
- définir des **contraintes min/max**,
- manipuler les **flexbox**,
- et adapter l'affichage selon la taille de l'écran.

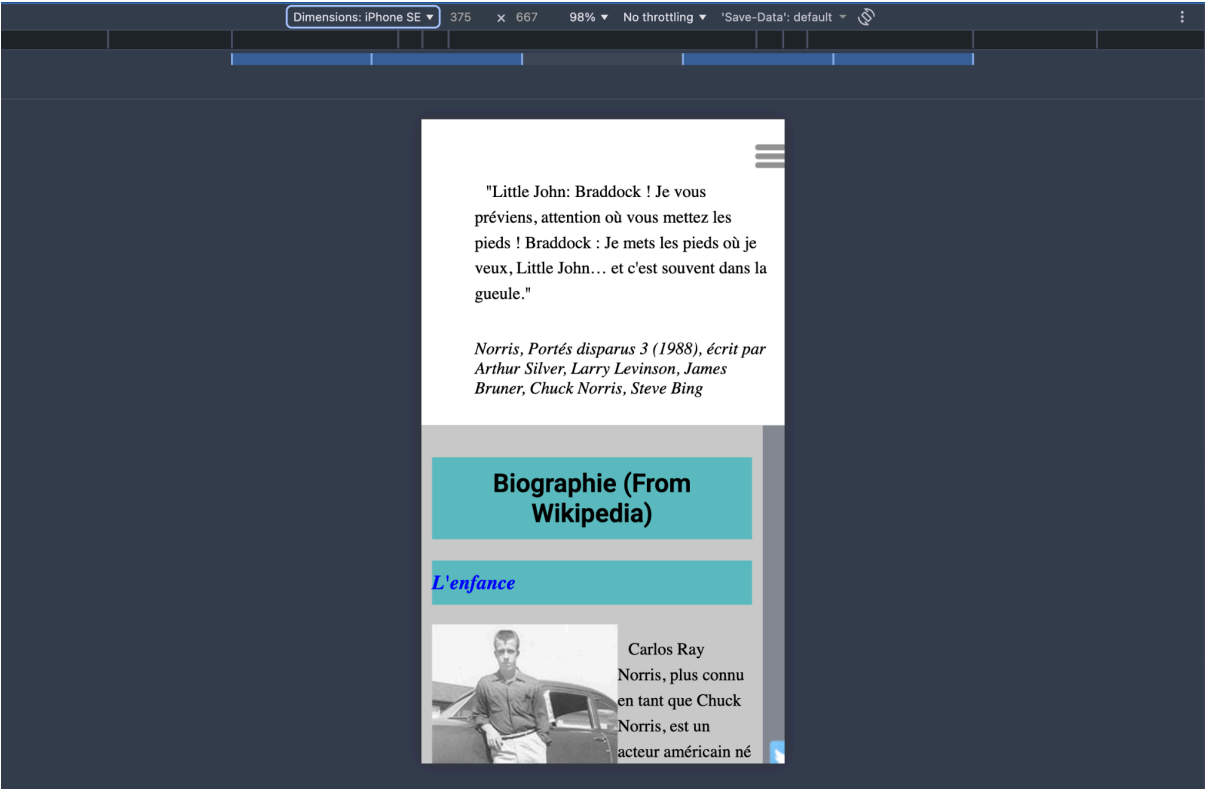
J'ai également compris l'importance de la balise **viewport** et des **menus adaptés aux mobiles**.

Grâce à ce TP, je peux désormais concevoir un **site web moderne, fluide et compatible avec tous les supports**.

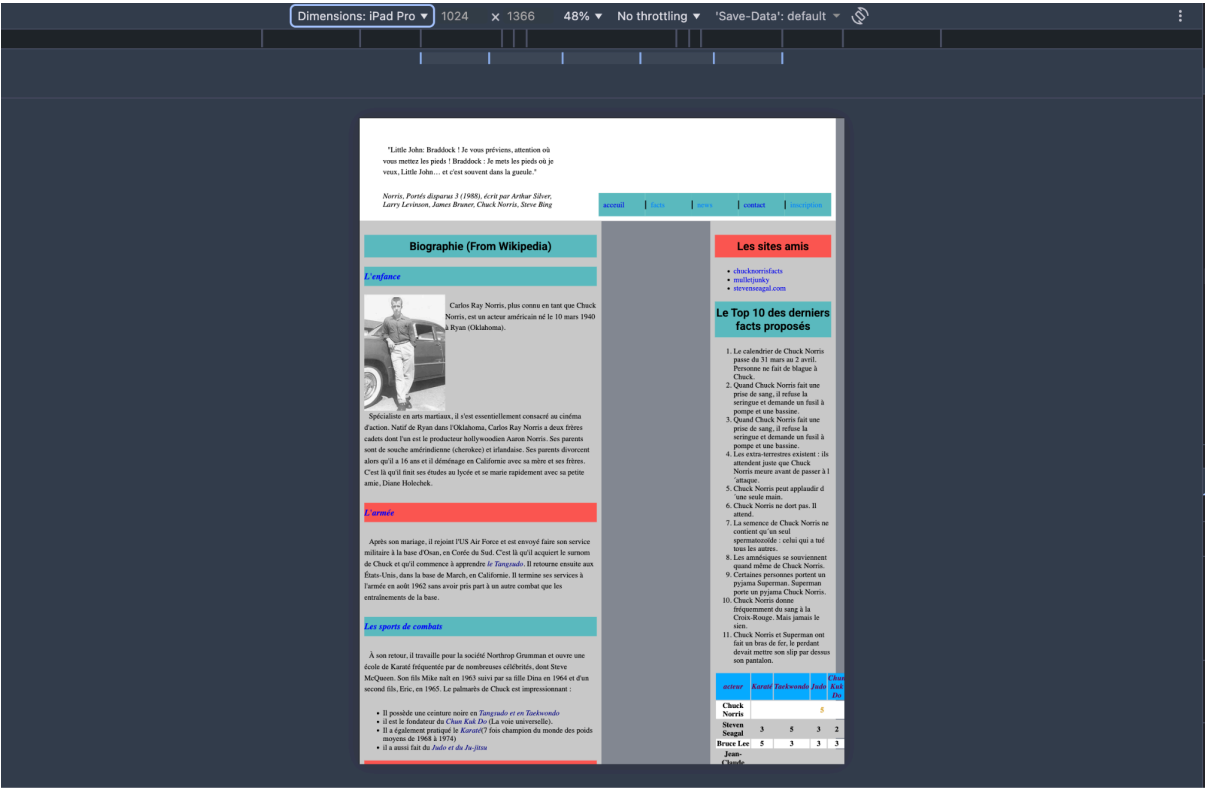
4. Résultat

Le site de Chuck Norris est maintenant **totalement responsive** :

- il s'adapte à la taille de l'écran,
- le menu devient un burger sur mobile,
- les tableaux sont masqués sur petit écran,
- et les blocs <article> et <aside> se réorganisent automatiquement.



site sur iphone se



site sur ipad pro

5. Annexe code CSS final

```
1  /* ----- STYLES GÉNÉRAUX ----- */
2  body {
3      background-color: #838892;
4      width: 100%; /* largeur demandée */
5      margin: auto; /* centre le body dans la fenêtre */
6  }
7  main {
8      display: flex; /* transforme main en conteneur flex */
9      flex-direction: row; /* met les enfants côte à côte */
10     justify-content: space-between; /* laisse de l'espace si nécessaire */
11 }
12 article {
13     width: 300px;
14     max-width: 500px;
15     min-width: 200px;
16     flex-shrink: 2; /* rétrécit 2x plus vite qu'un aside */
17     flex-grow: 3; /* gagne 3 parts d'espace */
18     background-color: #cccc;
19     padding: 10px;
20 }
21
22 aside {
23     width: 200px;
24     max-width: 250px;
25     min-width: 150px;
26     flex-shrink: 1; /* rétrécit moins vite */
27     flex-grow: 1; /* gagne 1 part d'espace */
28     background-color: #cccc;
29     padding: 10px;
30 }
31
32 h2 {
33     text-align: center;
34     font-family: "Roboto";
35     font-weight: 600;
36 }
37
38 h2, h3 {
39     padding: 10px 0;
40 }
41
42 p {
43     line-height: 150%;
44     text-indent: 10px;
45     margin: 30px 0;
```



```

42  p {
43  }
44  }
45  }
46  }
47  }
48  h3 {
49      font-style: italic;
50      color: blue;
51      border: 1px solid #CCCCCC;
52  }
53  }
54  h2.pair, h3.pair {
55      background-color: #5B8DBF;
56  }
57  }
58  h2.impair, h3.impair {
59      background-color: #FF5858;
60  }
61  }
62  .Spécialiste {
63      clear: left;
64  }
65  }
66  .skill {
67      font-style: italic;
68      color: navy;
69  }
70  }
71  a {
72      text-decoration: none;
73  }
74  }
75  a:visited {
76      color: #0088FF;
77  }
78  }
79  a:hover {
80      color: orange;
81  }
82  }
83  thead {
84      background-color: #00AAFF;
85  }
86  }
87  th .skill {
88      color: #640051;
89  }
90  }

```

```

91  table tbody:nth-of-type(even) tr {
92      background-color: #cccc;
93  }
94  }
95  table tbody:nth-of-type(odd) tr {
96      background-color: white;
97  }
98  }
99  .chuck {
100      color: goldenrod;
101  }
102  }
103  table.skill {
104      padding: 0 5px;
105  }
106  }
107  .jeune {
108      float: left;
109  }
110  }
111  .beware {
112      height: 300px;
113      float: right;
114  }
115  }
116  /* ----- ICÔNES DE RÉSEAUX ----- */
117  .reseaux {
118      position: fixed;
119      bottom: 40px;
120      right: 10px;
121      width: 40px;
122      margin-bottom: 5px;
123  }
124  }
125  .reseau {
126      position: fixed;
127      bottom: 0;
128      right: 10px;
129      width: 40px;
130      margin-bottom: 5px;
131  }
132  }
133  /* ----- MENU PRINCIPAL ----- */
134  nav {

```

```

133  /* ----- MENU PRINCIPAL ----- */
134  nav {
135    margin: right;
136    height: 50px;
137
138    /* Pour vérifier le placement à droite */
139    background-color: transparent;
140
141    display: flex;
142    flex-direction: row;
143    justify-content: flex-end; /* aligne tout le menu à droite */
144    align-items: stretch; /* les div prennent la hauteur complète */
145  }
146
147  nav > div {
148    display: flex;
149    width: 100px;
150    background-color: #508080;
151    position: relative; /* nécessaire pour positionner les sous-menus */
152    align-items: center; /* centre verticalement le texte */
153  }
154
155  nav a {
156    background-color: transparent;
157    padding: 0 10px;
158    border-left: 2px solid black;
159    display: block; /* meilleure surface cliquable */
160  }
161
162  .puce {
163    display: inline-block;
164    width: 6px;
165    height: 6px;
166    border-radius: 50%; /* forme circulaire */
167    background-color: black;
168    visibility: hidden; /* invisible par défaut */
169  }
170  nav .submenu div:hover .puce {
171    visibility: visible;
172  }
173  /* Supprimer la bordure du premier lien */
174  nav > div:first-child a {
175    border-left: none;
176  }
177  /* ----- SOUS-MENUS ----- */

```

```

178  .submenu {
179    display: none; /* masqué par défaut */
180    position: absolute;
181    top: 50px; /* décalage vertical sous le menu */
182    left: 0;
183    background-color: #f0f0f0;
184  }
185
186
187  nav div:hover .submenu {
188    display: block;
189    align-items: flex-end; /* visible au survol du parent */
190  }
191
192  header {
193    display: flex; /* active flexbox */
194    flex-direction: row; /* aligne la citation et le menu côte à côte */
195    justify-content: space-between; /* espace entre citation (gauche) et menu (droite) */
196    align-items: flex-end; /* aligne le menu en bas du header */
197    background-color: white; /* fond blanc demandé */
198    padding: 10px;
199  }
200
201  /* ----- FIN DU MENU ----- */
202  /* ----- BOUTON BURGER ----- */
203  .burger {
204    position: relative; /* base pour le menu interne */
205    display: flex;
206    justify-content: flex-end; /* place le bouton à droite */
207    align-items: center;
208    cursor: pointer;
209    visibility: hidden; /* caché par défaut */
210  }
211
212  #menu2 {
213    position: fixed; /* se place par rapport à la fenêtre */
214    top: 0;
215    right: 0;
216    width: 80vw; /* 80% de la largeur de la fenêtre */
217    height: 100vh;
218    display: none; /* masqué par défaut */
219    flex-direction: column;
220    justify-content: center;
221    align-items: center;
222    z-index: 1000;
223  }

```

